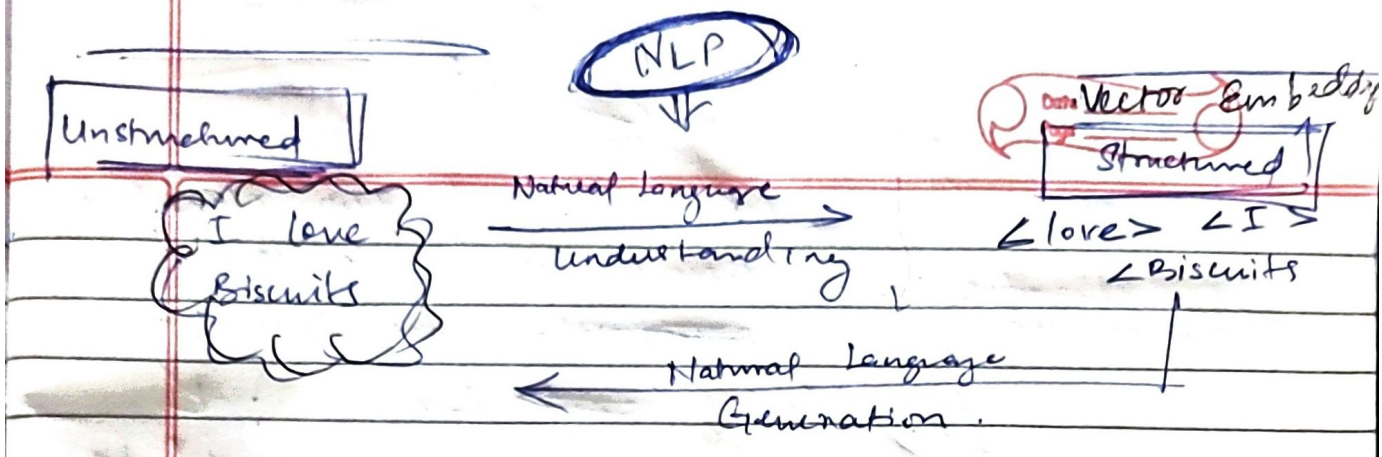
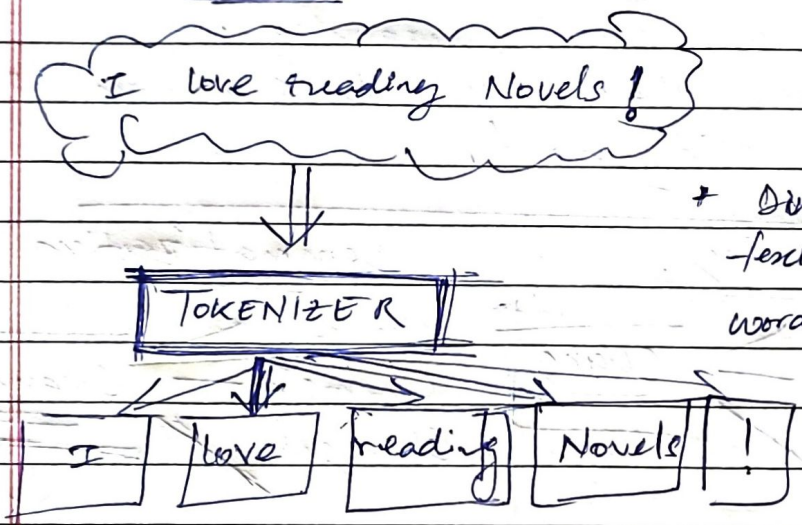


⇒ Steps Involved In NLP:

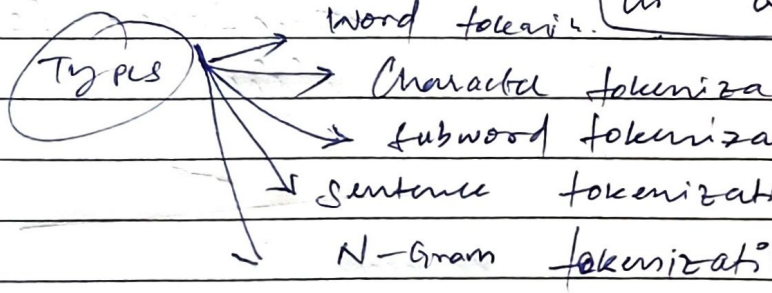
→ textpreprocessing
→ Feature Extraction
→ Modeling
→ Prediction (or) Output

→ Tokenization:



* Dividing a string or text into smaller words called tokens.

* A word is a token in a sentence. A character is a token in a word.



Add Biscuits ~~to~~ ~~and~~ shopping list & Chocolates ⇒ Segmentation → Add to shopping list, Add Biscuits and chocolates

Stop-words (~~not~~ these words are removed during the tokenization process)

Stemming : meaning → originate in (or) caused by

Skipping, Skipped, Skips → skip ← ing, ped, s.

→ (Removes last few characters from a word to its base form.)

→ Lemmatization :-

Eg: "Caring" ⟹ Car

It could be a valid or invalid word.

∴ Lemmatization :

- Uses POS (Parts of Speech tags)
- Considers words meaning.
- Ensures that the base form is a valid word.

Stemming	Lemmatization
Changing, Change, Changes → Chang	Changing, Changed, Changes → Change
⇓ not valid	⇓ valid word.
non-dictionary word	
Time-taken is lower	Time taken is higher

→ NER (Named Entity Recognition)

Person, Place, Location, Name

⇒ NLP pipeline :-

Input → Preprocessing → Feature Extraction → Modelling → Prediction / Output

Tokenization, Stop-word removal, Stemming / Lemmatization

Machines can't read words; thus, they are converted to vectors embeddings.

03/ Feb/ 2026 — (Sri class

Tagging

1. The / Student / went / to / class /
2. plays / well / with / others
3. Fruit / flies / like / a / banana.

DT : Determiner
NN : Noun
VB : verb
P : preposition
adverb

VB : 1. went, 2. Plays, 3. Flies

NN : Student, class, ...

DT : The // NN : Student // VB : went

(complete...)

⇒ Problem:

~~Find tagging error~~ in each of the following.

a) It is a nice night.

It/PRP is/VB a/DT nice/JJ night/NN

b) This crap game is over a garage in fifty-second street.

This/DT crap/NN is/VB over/IN a/D garage/NN in/IN fifty second/JJ street/NNP.

c) Nobody ever reads the newspaper she sells.

Nobody/NN ever/RB reads/VB the/DT newspapers/NNS (plural) she/Preposition sells (VB).

* Jane saw will
Noun verb Noun

* Jane will sleep
Noun modal Verb

⇒ Parts of speech Tagging

	Noun	Verb
Jane	1	0
saw	0	1
will	1	1
sleep	0	1

Unigrams

⇒ Jane will see will
⇒ Many will see Jane
⇒ will will see many

← Our Data.

	Noun	Verb	Modal
Jane	2	0	0
will	2	0	3
see	0	3	0
Many	2	0	0

(BIGRAMS)

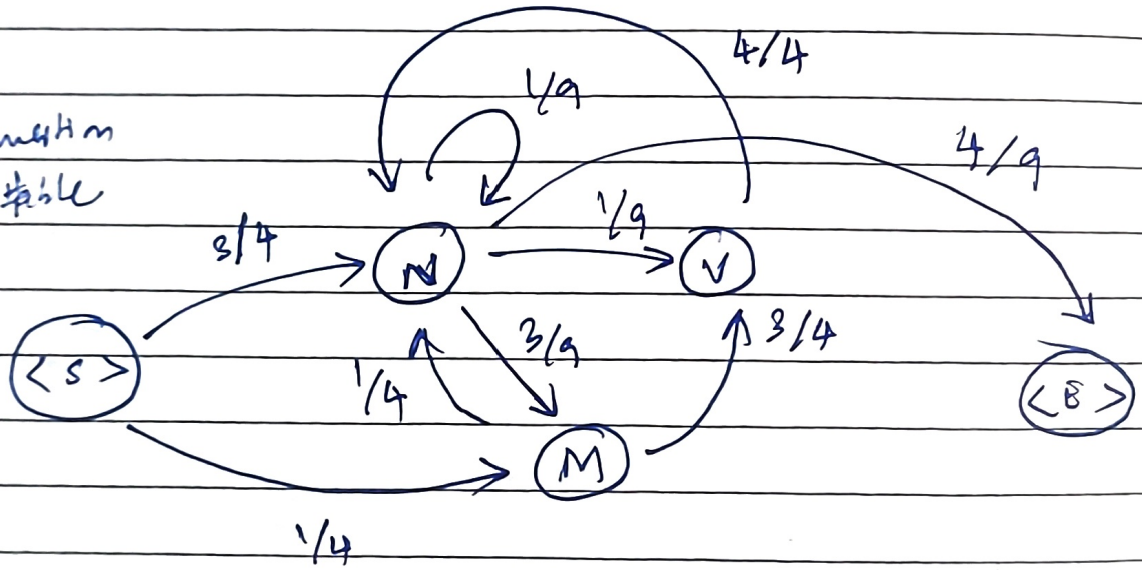
⇒ Bigrams:

	N-M	M-V	V-N
Jane-will	1	0	0
will-see	0	3	0
See-Jane			
will-will			
See-Many			
Jane-will			
See-will			

⇒ refer ppt.

	N	M	V	<E>
<S>	3/4	1/4	0	0
N	1/9	3/9	1/9	4/9
M	1/4	0	3/9	0
V	4/4	0	0	0

Transition table



ACTIVITY - 3

23BAI10917 — 12.07.2026

VIJAY RAJESH R. — Emotional Intelligence

1) Meanings:

* I'm OK / You're OK
  → Mutual respect & healthy collaboration.
* I'm not OK / You're OK
  → Feeling superior, critical, or blaming of others.
* A sense of hopelessness.

2)
i) 5
ii) 2
iii) 4
iv) 1
v) 3
vi) 3
vii) 3
viii) 1

3) i) I believe I have values as a person.

ii) Almost ~~everything~~ everytime.

4) i) It does not.
ii) I may feel overwhelmed on my toughest days but I say 'I can do this All day!'.
iii) I'm OK > You are Okay.

5) I'm OK, you're OK.
i) I must not give up.
ii) I must have consistency in whatever I do.
iii) Never Give up.

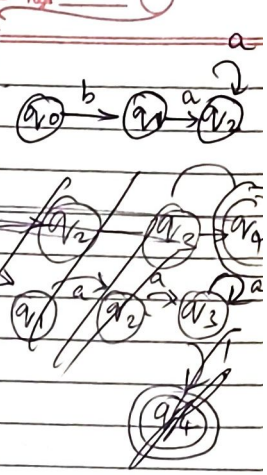
2/3/H IST

15/01/26.

Module - 1

1) FSA to recognise sheep talk

baa!
baaaa!
baaaaa!
- - -



for no. of states $n+1 = 2+1 = 3$

min length 1 + 1 NDFA

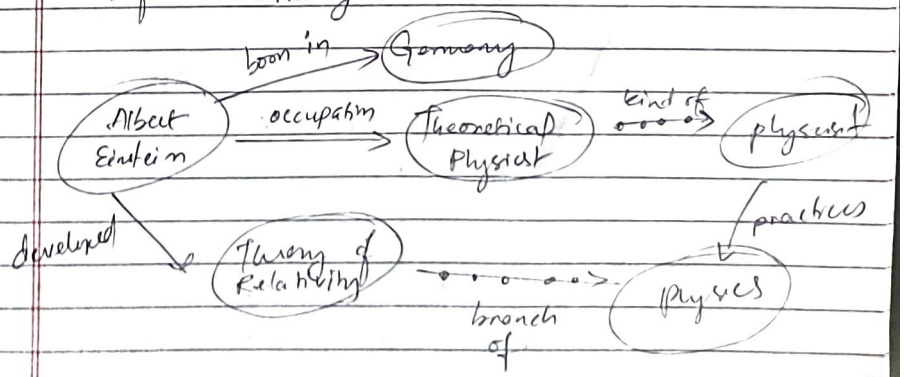
n

2) Ambiguity: (If n(parsetrees) > 1 : Grammar G is ambiguous)

- Syntactic
- Lexical
- Semantic
- Anaphoric
- Pragmatic

3) Knowledge Graph:

"Albert Einstein was a German-born theoretical physicist who developed theory of relativity.



pg-55/106.

⇒ Regular Expression → Refer studies (40 ~~70~~)

/a*/ → any string of ~~0~~ zero or more a's.

0, /a/, /aa/, /aaa/.

⇒ ~~Lect-3 ppt~~ : Lect-3 ppt!
Basics of TOC only

Module-2

(FSA-ppt) (tot: 34 pages)

Q. Write the Morphological parsed o/p:

Word	Morphological parsed o/p
Sheep	Sheep + N + SL
cats	Cat + N + PL
Walking	Walk + Verb + Present-Past
mice	Mouse + ~~Verb~~ N + PL

Eg: Present participle → indicates ~~an~~ going action.

Eg: Jessica found meditating on the rooftop.

⇒ pg-32/34:

Errors of Commission	Errors of Omission
organization - Organ	European - Europe
doing - doe	noise - noisy
going - police	Analysis - Analyse

(~~tot~~ tot: 38 pages)

⇒ PPT-2 : Morphology & Finite ~~State~~ ~~structures~~ Transducers.

Morphology: The study of the way of how words are built up using "morphemes"

~~Study~~ meaning smaller bearing-units.

Morpheme: ~~Single~~ Minimal meaning-bearing unit in a language.

Eg: Cat → Dogs
↓ ↓
~~Sin~~ Single morpheme Dog + s
Two Morphemes

Types:
- One morpheme – Nation
- Two Morpheme – National (nation, al)
- Three Morpheme – Nationalize (Nation, al, ize)
- Four Morpheme – Denationalize (de, national + ize)

⇒ Bounded Morphemenes:
It give 'meaning' when added to a morpheme.
→ s in walks.
→ re in replay.
→ er in cheaper.

⇒ Free morphemes:
↳ Acts alone as word.
→ Walks, ~~Spe~~ Speak, Ran

Circumfix: (en) lighten : (em) bolden (en)

Dt: 18/3/25

Two broad classes of ways to form words from morphemes

Inflection | Derivation

→ Inflection: ("Same-word" Modifier)
- Usually happens at the end (Suffix)
- Just adding an ending to word without changing the core meaning of the word.

eg: Walk + ing = Walking
Cat + s = Cats

- It is more productive than Derivation

→ Derivation: (The "Word Maker")
- To create entirely new word. This often changes the parts of the speech.

eg: Sing + er = Singer
Happy + un = Unhappy

- It happens at both Prefix and suffix
- It is less productive than Inflection.
- Very complex.

Module-2

Syntax Parsing:

A meaning bearing unit in a language (walks, talks, Table, Teach)

Morphemes

Free Morphemes
- stands alone as complete words
eg: Car, Table, Book

Bound Morphemes
- Must be attached to a free morpheme to be meaningful.
eg: (adj) Kind + ness = Kindness (Noun).

Types

Lexical Morphemes
- Carries Main meaning (Content words)
eg: Car, Book, Table

Functional Morphemes
- Shows Grammatical Relationships
eg: In, out, But, a, The like, prepositions.

Derivational Morphemes
- Changes meaning (or) POS of a word
eg: Happy → Unhappy

Inflectional Morphemes
- Doesn't Change Anything. It shows grammatical info of a word like → tense, plural, comparisons
eg: Cat + s → plural
Tall → comparisons

NLP

(1). NLP — Types (2) → Natural Language Understanding
→ Natural Language Generation

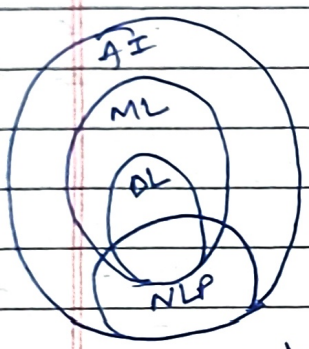
Ex: "How are you"

Natural language → NL Understanding → NL Generation

(Unstructured) (Converting the unstructured) (Converting the internal thoughts of machine to human readable form)

(Human → machine)

chatbots



Ex: "How are you"

(2) Metrics used in NLP : (refer notes in upcoming)

→ Precision
→ Accuracy
→ Recall
→ F1-score.

① Edit Distance (a.k.a Levenshtein distance)
② Euclidean Distance
③ Cosine similarity
④ Jaccard Index.

① Edit distance :

Algorithm for estimating similarity b/w two "string" values by performing minimum no. of operations to convert one value to the other.

② Euclidean Distance :

Compares the shortest distance b/w two objects :

If distance = 0, then objects are identical.

~~midterm~~

~~Important Questions~~

for two coordinates $(x_2 - x_1)^2 + (y_2 - y_1)^2$

~~$d = \sqrt{(x_2 - y_2)^2 + (x_1 - y_1)^2}$~~

~~Module 1~~

(1) formula: $d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$

d ↑ distance

(2) Cosine Similarity:

To determine the angle between the two objects.

$$Sim(A, B) = \cos\theta = \frac{A \cdot B}{|A|\,|B|}$$

value = 0 to 1

range

0 = similar

1 = Dissimilar

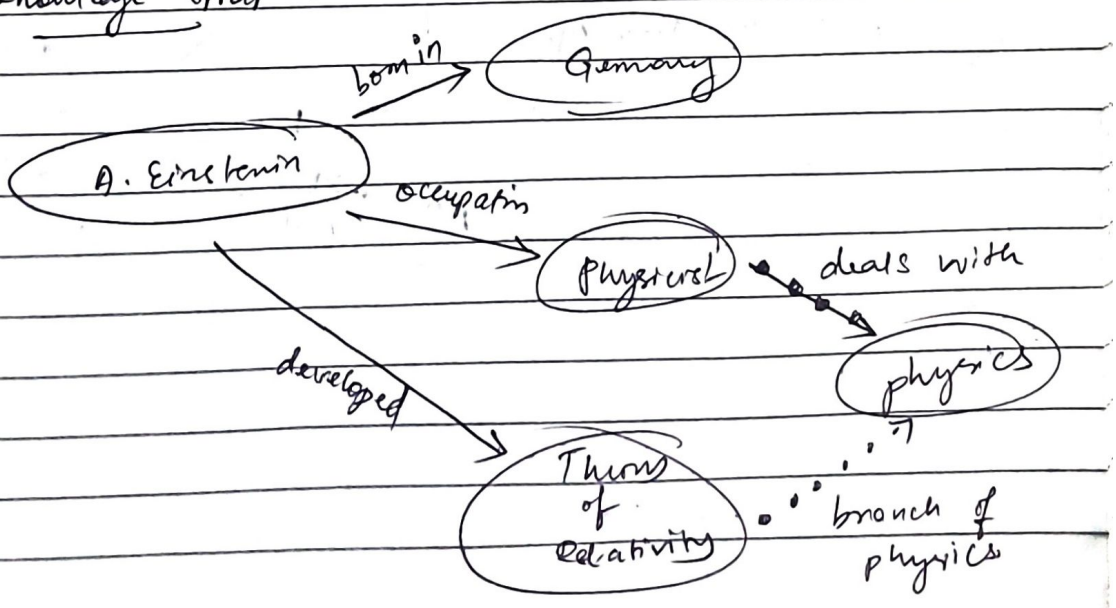
(3) Jaccard Index:

Measurement refer to no. of common words over all words.

$$\text{Jaccard Similarity} = \frac{A \cap B}{A \cup B}$$

If two are more common, then they are similar.

(4) Knowledge Graph:



~~Q.10 &~~

Q.7, 10 → Refer previous notes & onenote.

(12) POS tagging :-

- Labelling words with their Grammatical roles.
- Used in Machine Translation.
- " " Speech Recognition, Chatbots, and NLP systems.

Types of pos tagging →

- Transformation Based tagging
- Rule-Based Tagging
  - ↓
  - uses Grammatical rules to tag the words.
  - → But hard to handle Ambigous words
  - • No need training dates
  - • Medium Accuracy.
- Statistical Tagging
  - ↓
  - Uses probability to tag ~~the~~ based on training data.
  - • Can handle Ambiguity better
  - • Accuracy higher
- Neural/ML Tagging
  - ↓
  - Uses Neural Network & ML, DL, Transformers (like BERT) to learn patterns automatically from large data
  - • Accuracy very higher.
  - • Handles Ambiguity better than others

not in the syllabus

(6) **Regex:**

* used for pattern matching or comparing data with data

	Meaning:
[abc]	a, b or c
[^abc]	Any char. except a, b, c.
[a-z]	a to z
[A-Z]	A to Z
[a-z A-Z]	a to z, A to Z
[0 to 9]	0 to 9.

Quantifiers:

[ ] ?	occurs 0 or 1 time
[ ] +	occurs 1 or more time
[ ] *	occurs 0 or more times
[ ] {n}	occurs 'n' times
[ ] {n,}	occurs 'n' or more times
[ ] {y, z}	occurs atleast 'y' times but less than 'z' times.

Regex Metacharacters:

\d	[0-9]
\D	[^0-9]
\w	[a-z A-Z 0-9]
\W	[^\w]

This is negation of this

\ - tells computer to treat the next following characters as search character:

eg: \d, \w.

\- - tells computer to include '-' this as a literal character

escape

Entry

Example Problems:

(1) Mobile Number start with 8 or 9. with a total of 10 digits.

Solution:

[89] [0-9] {9}

(8) First character uppercase, contains lowercase characters, only one digit allowed is between.

Sol:

[A-Z] [a-z] * [0-9] [a-z] *

(9) Email-Id: Vijay123@gmail.com

~~[A-Z] [a-z] [0-9]~~

[A-Z a-z 0-9 _ \- \.] +

+ [@] [a-z] + [\.] [a-z]

@gmail

{2,3}

.com

.in

=2

explanation:

[A-Z a-z 0-9 \-\ .]

Escape character → The hyphen (-) is put inside a (\ \) Backslashes 'cuz (-) has predefined functions. In order to include (-), we use \-\ to tell the computer to take it as literal.

⇒ Sheeptalk: To demonstrate or explain → Regular languages → FSA

total no. of states: n+1

n = 2+1

n = 3

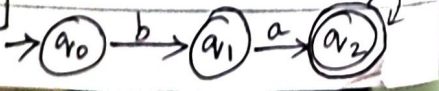
bal

baal

baaa!

Regular Expression: bat

FSA:



for NFA of sheep talk:

(9) FSA vs FST: → Transducer

Automaton

FSA	FST
To Accept or reject input.	To transform the input
No o/p produced	o/p is produced.
Used in Pattern Matching	used In Morphology, Translation.
i/p → process → Accept/Reject	i/p → process → output (o/p) Generated
In short, it is used for deciding to accept/reject a string.	In short, it is used for a transformation of the given input string.

Eg: Go → Went
Play → played
Cat → Cats.

~~Sheep Talk~~:

→ Mod-3

→ POS tagging:

① The grand jury commented on it
DT Adj NN VB ~~PRP~~ preposition pronoun

Determiner (these introduces nouns)

② Book that flight
VB DT NN

③ Does that flight serve dinner?
VBZ DT NN VB NN

- VBZ → present → 3rd person

④ It is a nice night.

IT/PRP is/VB a/~~DT~~DT nice/Adj night/NN.

→ Lookup table:

① Jane will see will
Mary will see Jane
will will see Mary.

words	Noun	Verb	Modal
Jane	2	0	0
will	2	0	3
see	0	3	0
Mary	2	0	0

will's are verbs

POS tagging →
- Rule-Based
- Stochastic POS (SPOS)
- Transformation Based tagger (TPOS)

Rule based Tagging	Stochastic Tagging	Transformation-Based Tagging
Uses Grammatical rules for tagging	Uses probability from dataset	Uses init of tagging + Correction rules
No large dataset needed	Needs a large tagged dataset	Needs a tagged dataset to learn
Very simple	Medium complexity	Medium complexity
90–92% Accuracy	93–95% Accuracy	95–97% Accuracy

- It is like a hybrid of Rule-based & S-POS

- working of ~~TB~~-POS:

① : Sentence is taken, split into words
② : Applies basic tagging to new words. (i.e.) Applies initial tags.
③ : Compare it with corrected data from the dataset.
④ : Applies transformation rules to correct errors.
⑤ : Output.

Module 2:

5) FSA for English:

i) Nominal:- NOMINAL

* Nominal means a noun phrase

Nominal → (DT) (ADJ)* N

Determine is optional

Noun is compulsory

0 or more adjectives can be there

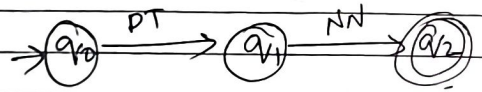
Transition Table:

For, The Dog:

State	I/P	Next state
→ q_0	DT	q_1
q_1	NN	* q_2

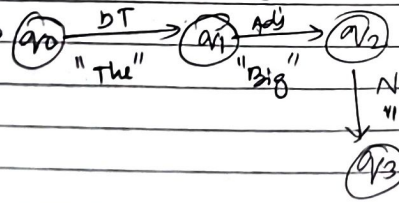
FSA:

The Dog:



Dog:

The Big dog:



7) Adjectives

Adjectives → (Adv) (Adj)*

↓ ↓

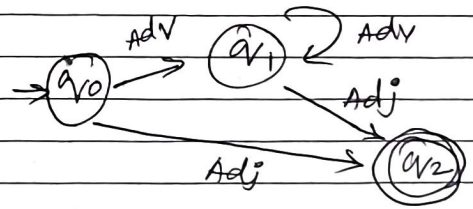
eg: Happy, Beautiful computing.

Adverb + Adjective :

Eg:
* Extremely Beautiful
* Very Happy.

Transition table:

Current state	I/P	next state
~~q_0~~	~~Adj~~	~~q_1~~
~~q_1~~	~~Adj~~	~~q_1~~
→ q_0	Adv	q_1
q_1	Adv	q_1
q_0	Adj	q_2*
q_1	Adj	q_2*

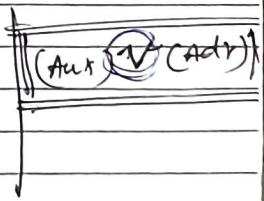


Verbs

Auxillary verbs → (is, have, will)

V → main verb.

Adv → Adverb



Transition table: (I have run quickly)

Aux V Adv

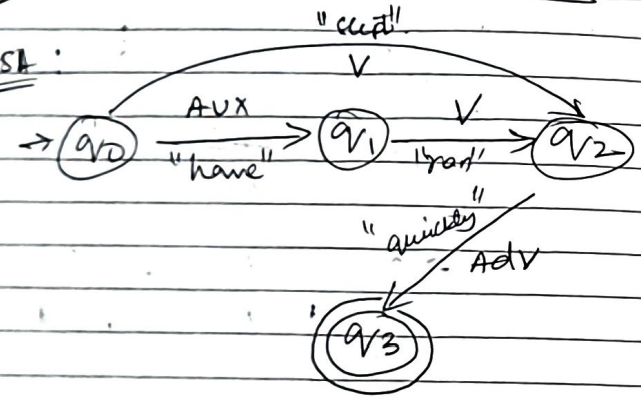
States:	I/P Aux	Next state
→ q_0		q_1

Eg: (have

Transition table :

Current state	i/p	Next state
→ q_0	AUX	q_1
q_1	V	q_2
q_2	Adv	q_3 ✱
q_0	V	q_2

FSA :



summary :-

Type	Pattern	Examples
① Nominal	(Det)+(Adj)+NN	✱ The Big Dog ✱ Dog ✱ Big Dog
② Adjective	(Adv)+Adj	✱ Beautiful ✱ Very & Happy
③ Verbs	(AUX)+V+(ADV)	✱ Have Ran quickly ✱ Run Quickly ✱ Run

Have will, want,

③ Metrics in NLP :

④ Accuracy = $\frac{\text{Correct Predictions}}{\text{Total Predictions}}$

⑤ Precision = $\frac{TP}{TP + FP}$

⑥ Recall = $\frac{TP}{TP + FN}$

⑦ F1 score = ~~Distance of~~ Balance of Precision & Recall.

Morphemes

Free Morphemes

Bound Morphemes

Lexical

Functional

Inflational

Derivational

carries main meaning

Book > Table > Laptop. Assignment

phrases relationship b/w one words

eg: On, Our, On, Inside

usually suffixes. Doesn't alter the meaning of the word.

eg: walking speaking

Word makes, Both positive and negative usually, Changes the meaning entirely

Happy + Un = Unhappy

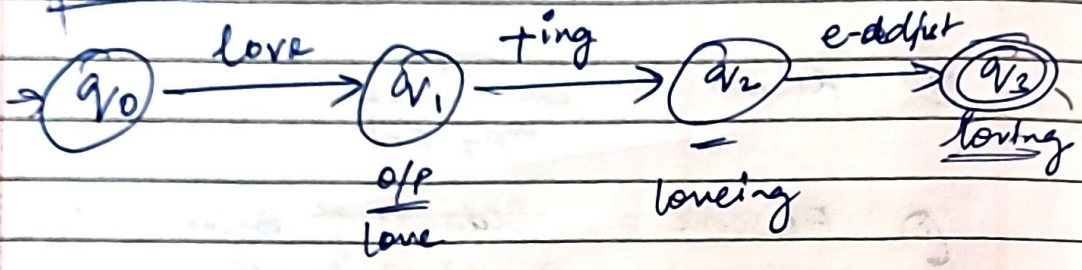
Sing + er = Singer.

⇒ e insertion, k insertion & y deletion.

It helps handling spelling changes when adding suffixes

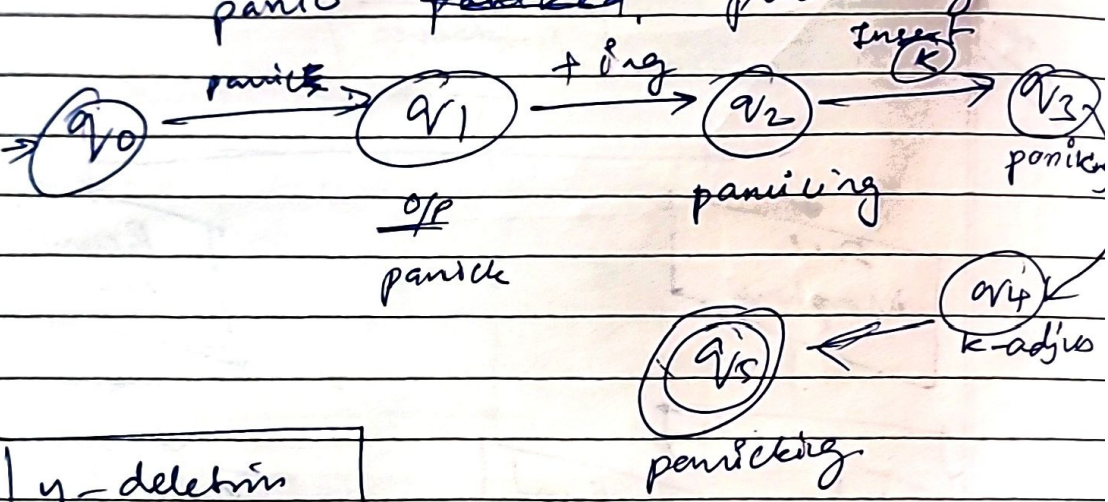
Sol

e - insertion eg: Love + ing



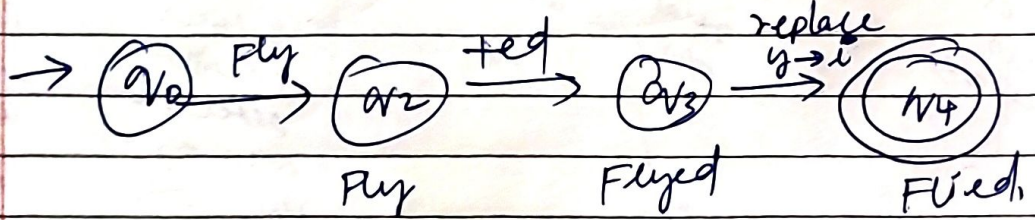
k - insertion

panic + ing = panicking.

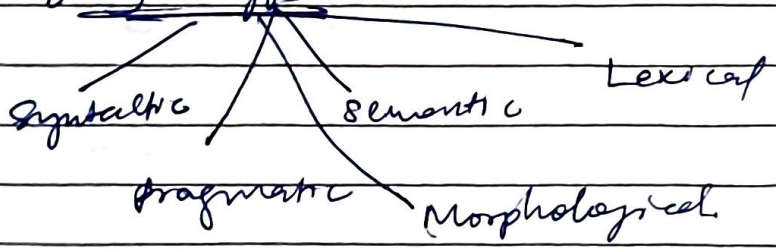


y - deletion

Fly + ed ⇒ Flied.



Ambiguity Types:



- TEE -

Module-1

(1) metrics in NLP :-

- cosine Similarity → ~~to find angle~~
- Euclidean Distance.
- Jaccard ~~Similarity~~ Index.

Cosine Similarity :

* To find angle (θ) b/w two objects.

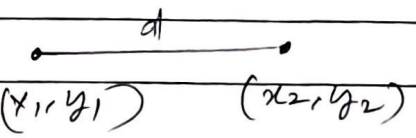
$$\cos\theta = \frac{A \cdot B}{|A|\,|B|}$$

Euclidean Distance: (Shortest Distance b/w two objects)

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

(or)

$$d = \sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$$



Jaccard Index :

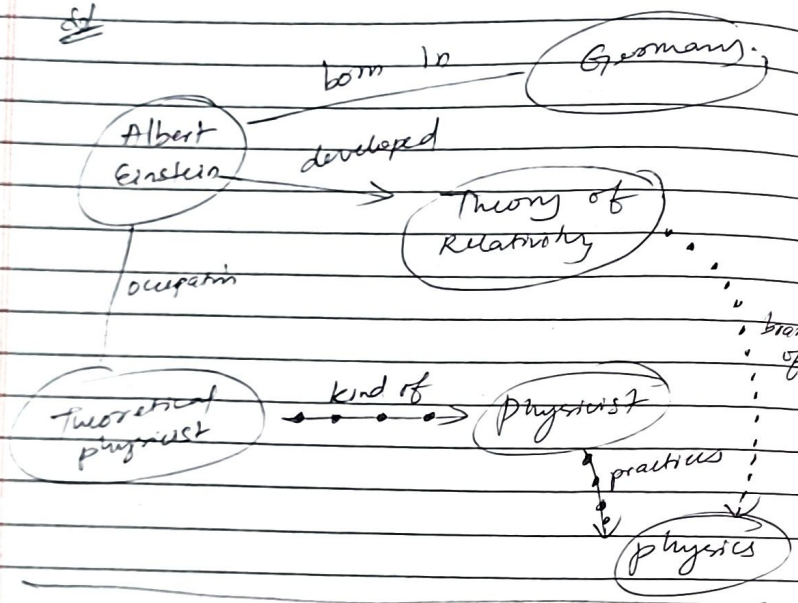
To find no. of common words to total number of words.

$$\text{Jaccard Similarity} = \frac{A \cap B}{A \cup B}$$

(2) Knowledge Graph :

* A way of storing data that resulted from an information extraction task.

Eg: Albert Einstein was a Germen-born theoretical physicist who developed theory of relativity.



(3) Regular Expressions: → Disjunction → Grouping → Precedence

i) Disjunction Operator:
- used to search for text.
- Pipe Symbol ("|")

Incorrect

/ puppy | ies /

↓

This will search for text either "puppy" or "ies" only, which is wrong

Correct

/ Pupp (y|ies) /

↑

This is correct

ii) Grouping:
It's just like using parantheses
Eg:

ba+ = baaaa.....

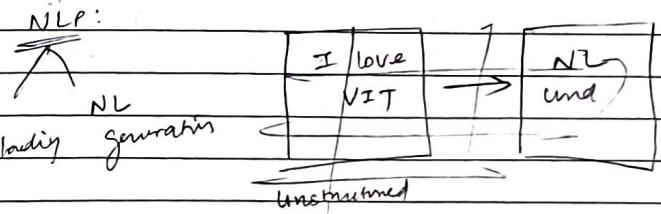
(ba)+ = bababa.....

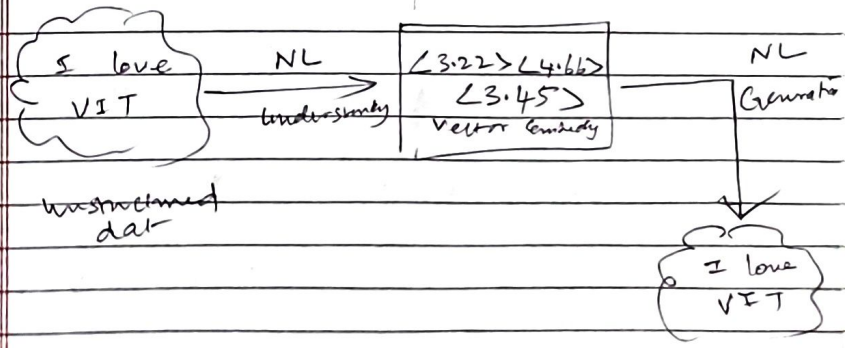
* allows us to repeat quantifiers like (*, +, {n}) to more no. of times

iii) Precedence:
* Its like rank or order of operations

Rank	Operation
(1)	Anything inside "( )" parantheses gets solved first
(2)	Next is Quantifiers (*, +, ?, {n})
(3)	Normal sequence {abc}
(4)	Disjunction (|)

(4)





Module-3

- Pos tagging
  - Rule-Based
    - uses grammatical rules
    - Accuracy 90-92%.
    - No need of dataset, as it uses rules of grammar
  - Stochastic
    - probabilistic approach
    - Accuracy 92-94%.
    - requires dataset
  - Transformation
    - Initial tagging + correction rules
    - Accuracy 94-97%.
    - requires dataset

Pos tagging:

The	Ground	Jury	commended	m	it
DT	Adj	NN	VBZ	PRP	PRP. (Pronoun)
(Determiner)				(progressive)	

Book	that	flight
VBZ	PRP / DT	NN

lovely — Adverb

Calendar

Module-4

Sentiment analysis:

Document	Sentiment
"I love this movie"	+ve
"this movie is great"	+ve
"I hate this movie"	-ve
"this movie is Terrible"	-ve

Python code:

```
data = [ . . . . ]

import re

def text_normalize(data):
    data = data.lower()                          # to remove numbers
    data = re.sub(r'\d+', '', data)
    data = re.sub(r'[^\w\s]', '', data)          # data
    data = re.sub(r'\s+', ' ', data).strip()
    return data

data = "I love my country"
normalized_data = text_normalize(data)

print(data)
print(normalized data).
```

re.sub(r"\d+", "", data)

r"\[^\W\s]", "", data

re.sub(r" \s+ ", " ", data).strip()

remove extra space / To remove punctuation

$P(tag/words) \propto P(words/tags) \times P(tags)$

Module - 5

Machine Translation (MT)

* Explores use of software to translate text or speech from 1 language to another.
* Machine Translation performs substitution of words in one language to other.
* Simple words:
  Automatic conversion of text/speech from one language to other.

Eg: The Crow is flying there!
* ತಿರುವೆ ಮೇಲೆ ಹಾರುತ್ತಿದೆ!

Usage: Google Translate, Bing Translator

→ Pushes boundaries of NLP due to the language divergence.

Basic Issues in Machine Translation:

* Ambiguity: Same meaning multiple words
  ನೀರು, ಜಲ, -All means water.

→ Linguistic Ambiguity:

→ Cultural & Contextual Nuance: Formal and Informal tone of speech

→ Grammatical Discrepancies: like Specific langu Verb-subject, Subject-verb agreements

→ Technical Limitations
* Hallucinations
* AI models doing error due to cultural insensitivity

---

(2) → Applications of NLP:

* AI chatbots
* Grammarly (Software)
* Content generation
* Fraud detection
* Machine Translation (Google Translate)
* Email Filtering.
* Virtual Assistant
* RAG, etc

⇒ Spell check & Summarization:-

→ Spell check:
* Process of identifying if a spelling is correct or not.
* Non-word error detection:-

```
If word in dictionary {
    okay, continue
}
else {
    Flag the word as Error
}
```

* Context-Error: Find error in the context
* Correction suggestion: Provides suggestion like Grammarly

⇒ Summarization:
* Gives summary of the original content.

⇒ Phrase Based Translation:

* It is a statistic-Machine Translation i.e., it uses probability
* It takes group of words and then starts translation rather than word by word translation. → (means - to dies)
* Eg: Kick the Bucket has different meaning in english; when it is translated to other languages, the words makes no sense.

So, here comes the Phrase-Based Translation.

Working principle:

I am eating watermelon.

Normal Translation: | I | am | eating | watermelon |

Phrase Based: | I am | eating watermelon |
phrase-1 phrase-2

Step-1: Break sentences into small group of words.
Step-2: Translate each phrase using learned data.
Step-3: Rearrange phrases to get correct meaning.

$P(A/B) = \frac{P(A) \cdot P(B/A)}{P(B)}$

⇒ Word Alignment in NLP:

* used to figure out which word in one language matches word in another language.

Eg: English: I drank Tea

Tamil: நான் தேநீர் குடித்தேன்

Alignment:

I → நான்
drank → குடித்தேன்
Tea → தேநீர்.

Mid-4:

⇒ FOPL

Components:
* Subject, Predicate
* Relationship
* Function

① All that Glitter is Gold.

$\forall x$ Glitter(x) → Gold

For all / For 'Every' = $\forall x$
For some / Atleast = $\exists x$

∀x → {All Man Drink coffee}

∀x Man → drinks (x, coffee)

∀x → All Birds Fly:

∀x Birds (x) → Fly (x),

→ Every Man respects his parent:

∀x Man (x) → respects (x, parent)

∃x → {Some boys are intelligent}

∃x boy (x) ∧ intelligent (x),

All Romans were either loyal to Caesar or hated him.

∀x: Romans (x) → loyal to (Caesar) ∨ hate (Caesar)

Main Connectives	
	∀x = →
	∃x = ∧

Marcus was a Man,
= Man (Marcus)

All people were Romans
= ∀x: People (x) → Romans (x).

Juliet loves Romeo!
= loves (Juliet, Romeo)

Rough

$$P(\text{Word}, \text{class}) = \frac{\text{count}(\text{word in class}) + 1}{\text{Tot words in class} + V}$$

V → vocabulary. all (contains unique words in the given dataset. No duplicates).

Program code:

```
import re

def normalize (text):          # lowercase.
    text = text.lower()
    text = re.sub(r"\d+", " ", text)
                  #Removes decimals
    text = re.sub(r"\W^\w\s", " ", text)
                                .strip()
    # Removes extra spaces punctuations
    text = re.sub(r"\s+", " ", text)
                  #Removes extra spaces.
    return text.

output = normalize(text)
print(output)
```